



گزارش کار پروژه

پروژه: پیاده سازی سمافور برای مسئله غذاخوردن فیلسوفها

درس: اصول سیستمهای عامل

استاد: دکتر اکرم سلطانی

تهیه شده توسط علی فاضل نیا

خرداد ماه ۱۴۰۴

دانشگاه شهید مدنی آذربایجان
Azarbaijan Shahid Madani University



مقدمه

باسمه تعالی

در دنیای سیستم‌های کامپیوتری مدرن، اجرای همزمان (Concurrency) نقشی حیاتی در افزایش بهره‌وری، کارایی و پاسخ‌گویی ایفا می‌کند. با گسترش معماری‌های چندپردازه‌ای و چندنخی، نیاز به راهکارهایی برای هماهنگ‌سازی دسترسی به منابع مشترک بیش از پیش احساس می‌شود. یکی از مفاهیم کلیدی برای مدیریت همگام‌سازی میان فرآیندها، سمافور (Semaphore) است؛ ابزاری قدرتمند که برای کنترل دسترسی فرآیندها به منابع اشتراکی بدون تداخل و بروز خطاهای رقابتی (Race Conditions) به کار می‌رود.

سمافور برای اولین بار توسط « ادسخر دیکسترا » در دهه ۱۹۶۰ معرفی شد و به سرعت به یکی از پایه‌های طراحی سیستم‌عامل‌ها و برنامه‌نویسی همزمان تبدیل شد. سمافور با دو عملیات اصلی خود، یعنی P (یا wait) و V (یا signal)، این امکان را فراهم می‌سازد که فرآیندها با نظم مشخصی به منابع محدود دسترسی پیدا کنند.

برای درک بهتر چالش‌های همگام‌سازی، مثال‌های کلاسیکی در ادبیات سیستم‌عامل مطرح شده‌اند که از میان آن‌ها، مسئله غذاخوردن فیلسوف‌ها (Dining Philosophers Problem) از شهرت و اهمیت خاصی برخوردار است. این مسئله توسط دیکسترا با هدف بررسی رقابت میان فرآیندها برای دستیابی به منابع مشترک طراحی شده و تاکنون در بسیاری از آموزش‌ها، پژوهش‌ها و حتی پیاده‌سازی‌های واقعی مورد استفاده قرار گرفته است.

در این گزارش کار، ابتدا به معرفی دقیق مفهوم سمافور، انواع آن و کاربردهایش خواهیم پرداخت. سپس مسئله فیلسوفان را شرح داده و روش‌های مختلف حل آن را بررسی می‌کنیم. در ادامه، پیاده‌سازی عملی این مسئله با استفاده از زبان Python و روش دایکسترا ارائه می‌شود تا به صورت عملی با چالش‌ها و راهکارهای همگام‌سازی آشنا شویم.

سمافور چیست؟

در سیستم‌های عامل و برنامه‌نویسی همزمان، یکی از چالش‌های اصلی، مدیریت صحیح دسترسی همزمان چندین فرآیند یا نخ (Thread) به منابع مشترک است. در صورت عدم هماهنگی، ممکن است وضعیت‌هایی مانند رقابت (Race Condition)، انسداد (Deadlock) یا گرسنگی (Starvation) به وجود آید که پایداری و عملکرد سیستم را تهدید می‌کند. به منظور حل این چالش‌ها، مفاهیم متعددی مطرح شده‌اند که یکی از مهم‌ترین و بنیادی‌ترین آن‌ها سَمافور (Semaphore) است.

تعریف سَمافور (از دیدگاه دایکسترا)

مفهوم سَمافور برای نخستین بار توسط دانشمند هلندی، ادسخر دایکسترا (Edsger W. Dijkstra) در دهه ۱۹۶۰ معرفی شد. از دیدگاه او، سَمافور یک متغیر مجرد است که می‌توان از آن برای کنترل دسترسی همزمان به منابع اشتراکی استفاده کرد. دایکسترا عملیات‌های روی سَمافور را با دو تابع اصلی معرفی کرد:

• $P()$ از کلمه هلندی Probere به معنای « امتحان کردن »

• $V()$ از کلمه Verhogen به معنای « افزایش دادن »

این توابع به ترتیب معادل با $wait()$ و $signal()$ در مفاهیم مدرن هستند.

انواع سَمافور

سَمافورها معمولاً به دو نوع کلی تقسیم می‌شوند:

۱. سَمافور دودویی (Binary Semaphore) :

که فقط دو مقدار ۰ یا ۱ را می‌پذیرد. معادل با مفهوم Mutex (قفل متقابل) در بسیاری از زبان‌های برنامه‌نویسی است و برای مدیریت همگام‌سازی‌های ساده بین دو نخ یا فرآیند کاربرد دارد.

۲. سَمافور شمارشی (Counting Semaphore) :

دارای مقدار صحیح غیرمنفی است و برای کنترل دسترسی به منابعی که به صورت محدود اما بیش از یکی موجودند (مثل یک صف چاپ یا چند چاپگر) استفاده می‌شود. مقدار اولیه سَمافور نشان‌دهنده تعداد منابع در دسترس است.

عملیات‌های اصلی $wait(P)$ و $signal(V)$:

سَمافورها بر اساس دو عملیات اتمیک (Atomic) کار می‌کنند:

• $\text{wait}(S)$ یا $P(S)$:

اگر مقدار سمافور S بیشتر از صفر باشد، یک واحد از آن کم می‌شود و فرآیند ادامه می‌یابد. در غیر این صورت، فرآیند در صف انتظار قرار می‌گیرد و مسدود می‌شود تا وقتی که مقدار سمافور مثبت شود.

• $\text{signal}(S)$ یا $V(S)$:

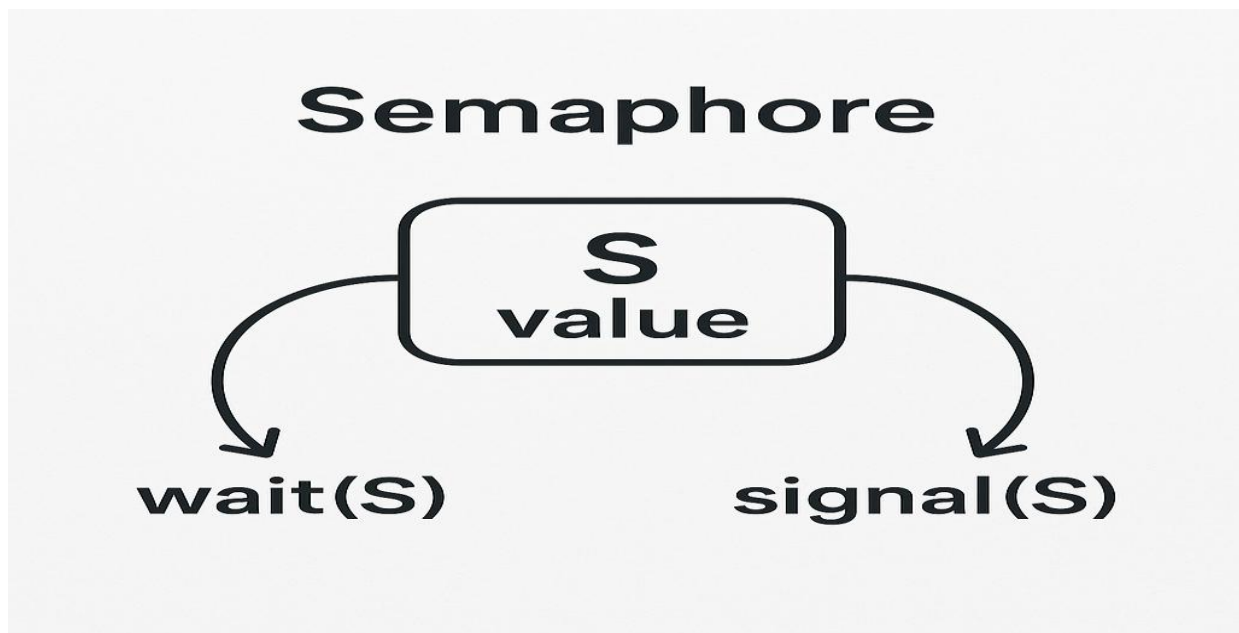
یک واحد به مقدار سمافور افزوده می‌شود. اگر فرآیندهایی در صف انتظار باشند، یکی از آن‌ها بیدار و فعال می‌شود.

این دو عملیات باید به صورت اتمی اجرا شوند تا از تداخل‌های ناخواسته جلوگیری شود. در برخی معماری‌ها، اجرای این اتمی بودن توسط سخت‌افزار مانند دستور (test-and-set) تضمین می‌شود.

کاربردها و مثال‌ها

سمافورها در طیف وسیعی از کاربردها مورد استفاده قرار می‌گیرند، از جمله:

- مدیریت منابع محدود: مانند دسترسی به حافظه مشترک، فایل‌ها، یا دستگاه‌های I/O
 - پیشگیری از بن‌بست: با طراحی صحیح ترتیب قفل‌گذاری منابع
 - همگام‌سازی نخ‌ها: در برنامه‌های چندنخی برای اطمینان از اجرای منظم
 - حل مسائل کلاسیک هم‌زمانی: مانند مسئله تولیدکننده-مصرف‌کننده (Producer-Consumer)، مسئله خوانندگان و نویسندگان (Readers-Writers) و مسئله غذاخوردن فیلسوف‌ها
- برای مثال، در مسئله تولیدکننده-مصرف‌کننده، یک سمافور شمارشی می‌تواند تعداد مکان‌های خالی یک بافر را بشمارد و به کمک $\text{wait}(S)$ و $\text{signal}(S)$ مانع از پر یا خالی شدن بیش از حد آن شود.



مسئله غذاخوردن فیلسوف‌ها

شرح مسئله

مسئله غذاخوردن فیلسوف‌ها (Dining Philosophers Problem) یکی از مسائل کلاسیک در حوزه همگام‌سازی و هم‌زمانی است که توسط ادسخر دایکسترا در سال ۱۹۶۵ معرفی شد. این مسئله به‌گونه‌ای طراحی شده تا چالش‌های مربوط به دسترسی هم‌زمان چند فرآیند به منابع محدود را مدل‌سازی کند. فرض کنید پنج فیلسوف در اطراف یک میز گرد نشسته‌اند. هر فیلسوف دارای دو وضعیت ممکن است: **تفکر و غذا خوردن**. بین هر دو فیلسوف، یک عدد چنگال وجود دارد و برای غذا خوردن، هر فیلسوف باید دو چنگال (سمت چپ و راست خود) را هم‌زمان در اختیار داشته باشد. پس از غذا خوردن، چنگال‌ها را کنار می‌گذارد و دوباره شروع به تفکر می‌کند. این مدل، به‌صورت نمادین رفتار چندین نخ یا فرآیند را که برای دسترسی به منابع اشتراکی تلاش می‌کنند، شبیه‌سازی می‌کند.

اهداف طراحی این مسئله

طراحی و بررسی راه‌حل‌های این مسئله، با هدف درک و مدیریت وضعیت‌های بحرانی زیر انجام می‌شود:

- **بن‌بست (Deadlock):** هنگامی که هر فیلسوف یک چنگال را بردارد و منتظر دیگری بماند، هیچ‌کدام نمی‌توانند به غذا خوردن ادامه دهند.
- **گرسنگی (Starvation):** زمانی که یک یا چند فیلسوف به‌طور مداوم موفق به گرفتن چنگال‌ها نشوند و هیچ‌گاه غذا نخورند.
- **تداخل (Race Condition):** هنگامی که عملیات هم‌زمان منجر به وضعیت ناسازگار یا ناخواسته می‌شود.
- **همگام‌سازی (Synchronization):** اطمینان از دسترسی ایمن و منظم به منابع محدود، بدون ایجاد اختلال یا اتلاف.

راه‌حل‌های مختلف

برای حل این مسئله، روش‌های متنوعی پیشنهاد شده‌اند که در ادامه به چند مورد کلیدی اشاره می‌شود:

۱. استفاده از Mutex (قفل متقابل)

در این راه‌حل، هر چنگال به‌صورت یک mutex (قفل دوجانبه) مدل می‌شود. فیلسوف ابتدا سعی می‌کند هر دو قفل (چنگال‌های چپ و راست) را به‌دست آورد. اگر نتواند، هر دو را آزاد کرده و دوباره تلاش می‌کند.

- مزایا: پیاده‌سازی ساده

- معایب: احتمال وقوع بن‌بست بسیار بالا، مگر با تغییر در ترتیب درخواست قفل‌ها

۲. استفاده از سمافور ساده

در این روش، هر چنگال یک سمافور دودویی است که فقط یک فیلسوف می‌تواند آن را در اختیار داشته باشد. فیلسوف برای غذا خوردن ابتدا `wait()` روی چنگال چپ، سپس روی چنگال راست اجرا می‌کند. پس از خوردن، `signal()` روی هر دو اجرا می‌شود.

- مزایا: کنترل بهتر نسبت به Mutex

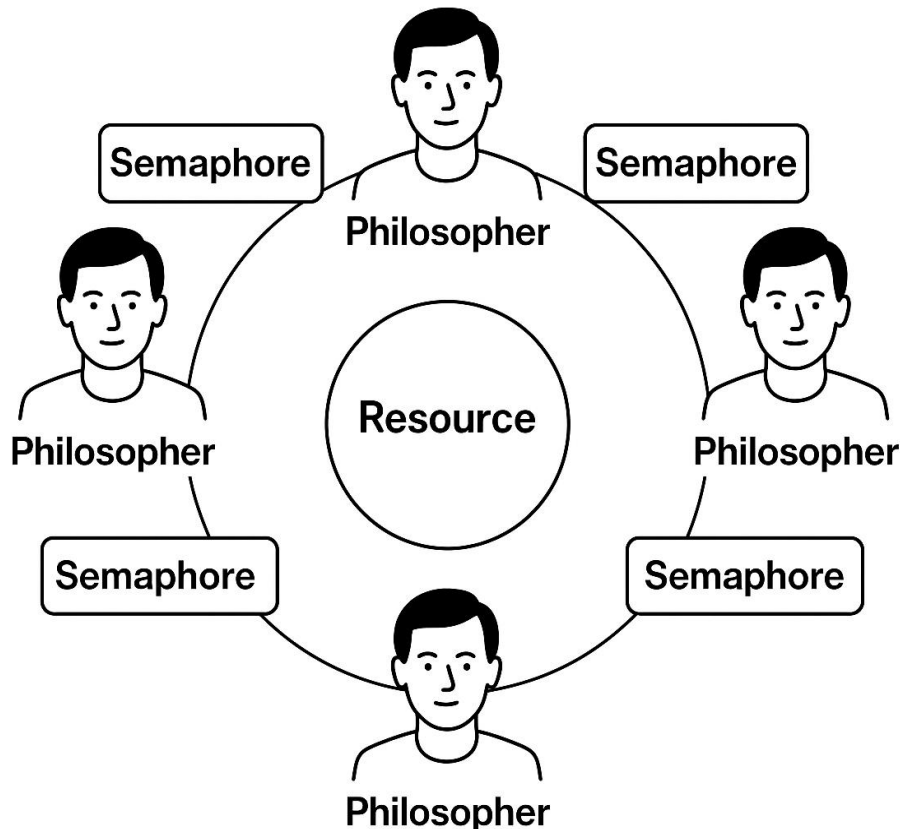
- معایب: همچنان بن‌بست ممکن است اگر تمام فیلسوف‌ها هم‌زمان چنگال چپ را بگیرند

۳. روش دایکسترا با استفاده از سمافور مرکزی

در این روش که توسط خود دایکسترا پیشنهاد شده، یک سمافور مرکزی (مثلاً با مقدار ۴ برای ۵ فیلسوف) تعریف می‌شود که تعیین می‌کند حداکثر هم‌زمان چند فیلسوف می‌توانند تلاش کنند تا چنگال‌ها را بردارند. پیش از گرفتن چنگال‌ها، فیلسوف باید `wait()` روی سمافور مرکزی انجام دهد، و پس از اتمام غذا خوردن، `signal()` اجرا شود.

- مزایا: پیشگیری کامل از بن‌بست، کاهش احتمال گرسنگی

- معایب: نیاز به سمافور سراسری و کمی پیچیدگی بیشتر



پیاده‌سازی با Python

پس از بررسی نظری مفاهیم مربوط به همزمانی (Concurrency) و مطالعه‌ی مسئله‌ی کلاسیک غذاخوردن فیلسوف‌ها، اکنون به پیاده‌سازی عملی این مسئله با استفاده از زبان Python و سازوکار سمافور می‌پردازیم. چالش اصلی، جلوگیری از شرایط بحرانی مثل بن‌بست (Deadlock) و گرسنگی (Starvation) در حین استفاده‌ی همزمان از منابع محدود است.

در این پیاده‌سازی، ما از روش داجیکسترا و سازوکار Semaphore برای هماهنگ‌سازی فعالیت‌های همزمان استفاده می‌کنیم. سمافور یک ساختار داده‌ی همزمان (Synchronization Primitive) است که امکان کنترل ورود همزمان چند نخ (Thread) به یک ناحیه‌ی بحرانی را فراهم می‌کند. به کمک سمافورها، می‌توان دسترسی به منابع مشترک را منظم و امن کرد. در این برنامه، از یک سمافور باینری به نام mutex برای محافظت از ناحیه‌ی بحرانی (که همان تغییر در وضعیت فیلسوف‌ها و بررسی وضعیت همسایه‌هاست) استفاده شده است. همچنین، برای هر فیلسوف نیز یک سمافور اختصاصی تعریف شده تا وضعیت انتظار یا مجاز بودن او برای غذا خوردن را کنترل کند.

تمام منطق برنامه در قالب نخ‌ها (Threads) پیاده‌سازی شده است؛ به‌گونه‌ای که هر فیلسوف در یک نخ مستقل قرار دارد و به‌صورت پیوسته بین سه وضعیت "تفکر" (Think)، "گرسنگی" (Hungry) و "غذا خوردن" (Eat) تغییر حالت می‌دهد. تابع take_forks زمانی اجرا می‌شود که فیلسوف قصد دارد غذا بخورد؛ این تابع وضعیت او را به "گرسنه" تغییر داده و بررسی می‌کند که آیا شرایط لازم برای غذا خوردن فراهم است یا خیر. اگر شرایط مناسب باشد (یعنی همسایه‌های چپ و راست در حال غذا خوردن نباشند)، اجازه‌ی ورود به ناحیه‌ی غذا خوردن به او داده می‌شود. پس از پایان غذا خوردن، تابع put_forks اجرا شده و وضعیت فیلسوف به "تفکر" تغییر می‌یابد، و همچنین بررسی می‌شود که آیا همسایگان او اکنون می‌توانند غذا بخورند یا خیر.

مزیت این پیاده‌سازی، سادگی، شفافیت و درعین حال وفاداری به منطق داجیکسترا است. با اجرای برنامه، کاربر می‌تواند وضعیت لحظه‌ای هر فیلسوف را مشاهده کرده و رفتار همزمانی بین نخ‌ها را به‌صورت تجربی درک کند. این مسئله نه‌تنها به‌عنوان تمرینی آموزشی بسیار مفید است، بلکه پایه‌گذار بسیاری از مفاهیم مهم در طراحی سیستم‌های عامل، مدیریت منابع اشتراکی، طراحی پروتکل‌های همزمانی و حتی کاربردهای توزیع‌شده به‌شمار می‌رود.

کد کامل پیاده‌سازی مسئله غذاخوردن فیلسوف‌ها با استفاده از سمافور روش (Dijkstra) در Python

```
1 import threading
2 import time
3 n = 5
4 state = []
5 for i in range(n):
6     state.append("Think")
7 mutex = threading.Semaphore(1)
8 s = []
9 for i in range(n):
10    s.append(threading.Semaphore(0))
11 def left(i):
12     return (i + n - 1) % n
13 def right(i):
14     return (i + 1) % n
15 def test(i):
16     if state[i] == "Hungry" and state[left(i)] != "Eat" and state[right(i)] !=
17 "Eat": state[i] = "Eat"
18     s[i].release()
19 def take_forks(i):
20     mutex.acquire()
21     state[i] = "Hungry"
22     print(f"philosopher{i} is hungry")
23     test(i)
24     mutex.release()
25     s[i].acquire()
26 def put_forks(i):
27     mutex.acquire()
28     state[i] = "Think"
29     print(f"philosopher{i} is thinking")
30     test(left(i))
31     test(right(i))
32     mutex.release()
33 def philosopher(i):
34     while True:
35         print(f"philosopher{i} is thinking...")
36         time.sleep(1)
37         take_forks(i)
38         print(f"philosopher{i} is eating...")
39         time.sleep(2)
40         put_forks(i)
41 threads = []
42 for i in range(n):
43     threads.append(threading.Thread(target=philosopher, args=(i,)))
44 for t in threads:
45     t.start()
46 for t in threads:
47     t.join()
```

در ادامه به توضیح کد خواهیم پرداخت. (عکس‌ها تهیه شده از محیط vscode هستند.)

بررسی پیاده‌سازی مسئله غذاخوردن فیلسوف‌ها با استفاده از سمافور روش (Dijkstra) در Python

کدی که برای پیاده‌سازی مسئله غذاخوردن فیلسوف‌ها در صفحه قبل آورده شد را به ۹ بخش اصلی تقسیم کرده‌ایم و در ادامه به بررسی دقیق هر کدام از بخش‌ها خواهیم پرداخت.

۱. وارد کردن کتابخانه‌های ضروری

```
1 import threading
2 import time
```

threading: این کتابخانه برای مدیریت نخ‌ها (threads) و استفاده از سمافورها در پایتون به کار می‌رود. بدون آن نمی‌توان عملیات همزمان‌سازی و اجرای موازی فیلسوف‌ها را پیاده‌سازی کرد.

time: این کتابخانه برای ایجاد تأخیر زمانی به منظور شبیه‌سازی بهتر رفتار فیلسوف‌ها (تفکر و خوردن) و قابل فهم شدن اجرای برنامه استفاده می‌شود.

۲. تعریف متغیرهای وضعیت فیلسوف‌ها

```
3 n = 5
4 state = []
5 for i in range(n):
6     state.append("Think")
```

$n = 5$: این مقدار تعداد فیلسوف‌ها را تعیین می‌کند. فرض بر این است که ۵ فیلسوف به دور یک میز نشسته‌اند.

state: این لیست (آرایه) برای نگهداری وضعیت هر فیلسوف استفاده می‌شود. هر خانه از این آرایه یکی از وضعیت‌های زیر را دارد:

"Think": فیلسوف در حال تفکر است.

"Hungry": فیلسوف گرسنه شده و به دنبال چنگال‌هاست.

"Eat": فیلسوف در حال خوردن غذاست.

در ابتدای برنامه، همه‌ی فیلسوف‌ها در حال تفکر هستند.

```
7 mutex = threading.Semaphore(1)
8 s = []
9 for i in range(n):
10     s.append(threading.Semaphore(0))
```

mutex: یک سمافور باینری (binary semaphore) برای کنترل ناحیه بحرانی است. تنها یک نخ می‌تواند وارد ناحیه بحرانی شود تا از شرایط رقابت (Race Condition) جلوگیری شود.

s[i]: آرایه‌ای از سمافورهای صفر برای فیلسوف‌ها.

وقتی فیلسوفی گرسنه است، منتظر می‌ماند تا شرایط مناسب برای خوردن فراهم شود.

سپس با release() در تابع test(i) اجازه خوردن به او داده می‌شود. (با ساختار تابع test در ادامه آشنا خواهیم شد)

در take_forks(i)، فیلسوف با s[i].acquire() تا رسیدن زمان مناسب منتظر می‌ماند. (با ساختار تابع take_forks در ادامه آشنا خواهیم شد)

۴. توابع کمکی برای یافتن همسایگان

```
11 def left(i):
12     return (i + n - 1) % n
13
14 def right(i):
15     return (i + 1) % n
```

این دو تابع با استفاده از محاسبه مدولار همسایگان سمت چپ و راست فیلسوف نام را مشخص می‌کنند.

ساختار دایره‌ای میز باعث می‌شود که فیلسوف اول (شماره ۰) همسایه‌ای در سمت چپ داشته باشد (فیلسوف ۴).

هدف اصلی: بررسی وضعیت همسایگان برای تصمیم‌گیری درباره اینکه آیا فیلسوف می‌تواند غذا بخورد یا نه.

۵. تابع test(i) - بررسی امکان غذا خوردن

```
16 def test(i):
17     if state[i] == "Hungry" and state[left(i)] != "Eat" and state[right(i)] != "Eat":
18         state[i] = "Eat"
19         s[i].release()
```

هدف اصلی این تابع، بررسی شرایط شروع غذا خوردن برای فیلسوف نام است.

اگر:

فیلسوف گرسنه باشد (`state[i] == "Hungry"`) و فیلسوف سمت چپ در حال غذا خوردن نباشد و فیلسوف سمت راست در حال غذا خوردن نباشد؛ آنگاه:

وضعیت فیلسوف به "Eat" تغییر می‌کند.

با `s[i].release()` اجازه‌ی ورود به بخش غذا خوردن به او داده می‌شود.

۶. تابع `take_forks(i)` - درخواست چنگال‌ها

```
20 def take_forks(i):
21     mutex.acquire()
22     state[i] = "Hungry"
23     print(f"philosopher{i} is hungry")
24     test(i)
25     mutex.release()
26     s[i].acquire()
```

این تابع زمانی فراخوانی می‌شود که فیلسوف گرسنه می‌شود.

گام‌ها:

۱) با `mutex.acquire()` ورود امن به ناحیه بحرانی انجام می‌شود.

۲) وضعیت فیلسوف به "Hungry" تغییر می‌کند.

۳) `test(i)` بررسی می‌کند آیا شرایط برای غذا خوردن مهیاست یا نه.

۴) خروج از ناحیه بحرانی با `mutex.release()`.

۵) `s[i].acquire()` فیلسوف را بلوکه می‌کند تا زمانی که اجازه‌ی خوردن داده شود (در صورت آزاد نبودن چنگال‌ها).

۷. تابع `put_forks(i)` - آزادسازی چنگال‌ها

```
27 def put_forks(i):
28     mutex.acquire()
29     state[i] = "Think"
30     print(f"philosopher{i} is thinking")
31     test(left(i))
32     test(right(i))
33     mutex.release()
```

زمانی که فیلسوف غذا خوردنش تمام می‌شود این تابع اجرا می‌شود.

گام‌ها:

(۱) ورود امن به ناحیه بحرانی با `mutex.acquire()`.

(۲) تغییر وضعیت فیلسوف به "Think".

(۳) بررسی می‌شود که آیا همسایگان می‌توانند اکنون غذا بخورند.

(۴) با `test(left(i))` و `test(right(i))` اگر شرایط مناسب باشد، به آن‌ها اجازه داده می‌شود.

(۵) خروج از ناحیه بحرانی با `mutex.release()`.

۸. تابع `philosopher(i)` - رفتار کامل فیلسوف

```
34 def philosopher(i):
35     while True:
36         print(f"philosopher{i} is thinking...")
37         time.sleep(1)
38         take_forks(i)
39         print(f"philosopher{i} is eating...")
40         time.sleep(2)
41         put_forks(i)
```

• این تابع چرخه‌ی رفتاری فیلسوف را شبیه‌سازی می‌کند:

(۱) فیلسوف ابتدا فکر می‌کند.

(۲) بعد از مدتی گرسنه می‌شود و چنگال‌ها را می‌گیرد.

(۳) اگر موفق به گرفتن هر دو چنگال شود، شروع به غذا خوردن می‌کند.

(۴) سپس چنگال‌ها را می‌گذارد و دوباره شروع به فکر کردن می‌کند.

این چرخه تا بی‌نهایت ادامه می‌یابد. هر فیلسوف به‌صورت یک نخ جداگانه اجرا می‌شود.

۹. ایجاد و اجرای نخ‌ها

```
42 threads = []
43 for i in range(n):
44     threads.append(threading.Thread(target=philosopher, args=(i,)))
45
46 for t in threads:
47     t.start()
48
49 for t in threads:
50     t.join()
```

در این بخش:

برای هر فیلسوف یک Thread ساخته می‌شود و تابع philosopher(i) را اجرا می‌کند.

با start() همه نخ‌ها به صورت موازی شروع می‌شوند.

join(): باعث می‌شود برنامه اصلی تا پایان همه نخ‌ها منتظر بماند (در اینجا هیچ‌گاه تمام نمی‌شود چون حلقه while True است).

در نهایت، این پیاده‌سازی با بهره‌گیری از سمافورها و کنترل دستی ناحیه بحرانی، موفق به مدیریت صحیح همزمانی بین فیلسوف‌ها شده و از شرایطی مانند بن‌بست و گرسنگی جلوگیری می‌کند. همچنین برای درک بهتر منطق برنامه، در دل کد فایل پایتون اصلی نیز کامنت‌هایی قرار داده شده که مراحل و اهداف هر بخش را به طور خلاصه توضیح می‌دهند.

در آخر به مشاهده و بررسی نمونه‌ای از اجرای برنامه در محیط Windows Powershell خواهیم پرداخت.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\University\Bachelor\Term 6\Fundamentals of Operating Systems> cd '..\Project 1\'
PS D:\University\Bachelor\Term 6\Fundamentals of Operating Systems\Project 1> python -m a.py
philosopher0 is thinking...
philosopher1 is thinking...
philosopher2 is thinking...
philosopher3 is thinking...
philosopher4 is thinking...
philosopher1 is hungry
philosopher1 is eating...
philosopher0 is hungry
philosopher2 is hungry
philosopher3 is hungry
philosopher3 is eating...
philosopher4 is hungry
philosopher1 is thinking
philosopher1 is thinking...
philosopher0 is eating...
philosopher3 is thinking
philosopher3 is thinking...
philosopher2 is eating...
philosopher1 is hungry
philosopher3 is hungry
philosopher0 is thinking
philosopher0 is thinking...
philosopher4 is eating...
philosopher2 is thinking
philosopher2 is thinking...
philosopher1 is eating...
philosopher0 is hungry
philosopher2 is hungry
philosopher4 is thinking
philosopher4 is thinking...
philosopher3 is eating...
philosopher1 is thinking
philosopher1 is thinking...
philosopher0 is eating...
philosopher4 is hungry
philosopher1 is hungry
```

تحلیل رفتار سیستم هم‌زمان (Concurrent):

۱) شروع با «thinking»:

بلافاصله پس از اجرای برنامه، تمام فیلسوف‌ها با وضعیت thinking... وارد چرخه‌ی زندگی‌شان می‌شوند. این وضعیت نشان می‌دهد که هیچ‌کدام هنوز وارد بخش بحرانی (critical section) نشده‌اند:

philosopher0 is thinking...

philosopher1 is thinking...

philosopher2 is thinking...

philosopher3 is thinking...

philosopher4 is thinking...

(۲) گرسنه شدن چند فیلسوف همزمان (Hungry):

به مرور برخی فیلسوفان وضعیت خود را به hungry تغییر می دهند. به طور مثال:

philosopher1 is hungry

philosopher2 is hungry

philosopher0 is hungry

در این لحظه ها، فیلسوف ها درخواست دسترسی به دو چنگال مجاور خود را ثبت کرده اند، اما هنوز در حال بررسی شرایط برای ورود به بخش بحرانی هستند.

(۳) اجازه برای «خوردن» (Eat):

اگر فیلسوف بتواند هر دو چنگال را در اختیار بگیرد (یعنی همسایه هایش در حال «خوردن» نباشند)، وارد حالت eating... می شود:

philosopher1 is eating...

philosopher3 is eating...

philosopher4 is eating...

این نشان دهنده ی آزاد بودن چنگال های مجاور برای آن فیلسوف است.

(۴) بازگشت به «thinking»:

پس از پایان غذا خوردن، فیلسوف وضعیت خود را به «thinking» برمی گرداند و چنگال ها را آزاد می کند:

philosopher1 is thinking...

philosopher3 is thinking...

philosopher0 is thinking...

این آزادسازی باعث می‌شود فیلسوفان همسایه که منتظر بوده‌اند بتوانند وارد بخش بحرانی شوند.

نکات قابل توجه:

عدم وجود بن‌بست (Deadlock) :

در هیچ لحظه‌ای تمام فیلسوف‌ها به طور دائم در حالت hungry باقی نمی‌مانند. یعنی بن‌بستی اتفاق نیفتاده و الگوریتم موفق عمل کرده است.

عدم گرسنگی دائم (Starvation) :

هر فیلسوف بالاخره نوبت خود را دریافت کرده و وارد حالت eating می‌شود. این نشان‌دهنده‌ی منصف بودن الگوریتم است.

این خروجی نمایانگر یک اجرای موفق و پایدار از راه‌حل Dijkstra برای مسئله Dining Philosophers است. تمام مراحل چرخه‌ی تفکر ← گرسنگی ← غذا خوردن ← تفکر به‌طور منظم دیده می‌شود و سیستم با استفاده از سمافور و نخ‌ها توانسته همزمانی را به‌خوبی مدیریت کند. همچنین همان‌طور که در کد نیز با کامنت‌ها مشخص شده بود، این الگوریتم با جلوگیری از شرایط بحرانی، سیستمی امن و قابل‌اعتماد را پیاده‌سازی می‌کند.

نتیجه‌گیری نهایی

در این پروژه، با استفاده از زبان برنامه‌نویسی پایتون و بهره‌گیری از مفهوم سمافورهای دایکسترا موفق شدیم یک شبیه‌سازی کامل و بدون بن‌بست از مسئله‌ی غذاخوردن فیلسوف‌ها پیاده‌سازی کنیم. در پیاده‌سازی این سیستم هم‌زمان، تلاش شد تا با طراحی منطقی توابع، استفاده‌ی ایمن از منابع (چنگال‌ها) تضمین شود و همزمانی بین نخ‌ها به‌درستی مدیریت گردد. خروجی نهایی اجرای برنامه نیز صحت عملکرد الگوریتم را به‌خوبی نشان می‌دهد: همه‌ی فیلسوف‌ها در بازه‌های زمانی مختلف موفق به خوردن غذا شده‌اند، بدون آنکه بن‌بست یا گرسنگی دائمی رخ دهد. به‌عنوان نتیجه، این پیاده‌سازی می‌تواند به‌عنوان یک نمونه‌ی موفق از کنترل همزمانی در سیستم‌عامل‌ها معرفی شود.

منابع و مراجع

Operating System Concepts by Silberschatz, Galvin, and Gagne (Book)

Modern Operating Systems by Andrew S. Tanenbaum (Book)

[Wikipedia - Semaphore \(programming\)](#)

[Wikipedia - Dining philosophers problem](#)

[GeeksforGeeks – Semaphore in OS](#)

[Youtube - Jacob Sorber Channel](#)